

Handwritten Explanation

Fix #1: Correcting search condition grouping

* Location:

→ TaskRepository.java, in searchTasks; the matching conditions were also corrected in db/queries/search-tasks.sql and db/oracle/task-search-package.sql.

* How I found it:

→ I traced the search query and checked SQL evaluates AND and OR, AND is evaluated before OR.

* Root cause and fix:

Old logic:

[(not archived AND title matches)]

OR

[(description matches AND (no status selected OR || status matches)]

→ This allowed a title match to bypass the selected status, or a description match to bypass the archived check.

Corrected logic:

not archived AND

↓

(title matches OR description matches)

↓

AND

*
(no status selected OR status matches)

→ Now either title or description can match, but every result must be unarchived and match the selected status, if one was selected.

* How I checked it:

→ Searching mobile with status OPEN returned task 1, where "mobile" appears in the description. The same search with status DONE ~~and~~ returned no tasks.

→ Searching legacy returned no tasks because its matching seed tasks are archived.

→ I also updated the Oracle reference query for consistency, but did not run it locally because the local ~~st~~ setup uses H2.

Fix#2: Removing Request Delay and Handling Invalid Inputs

* Location:

→ backend/src/main/java/com/internal/tasktracker/TaskController.java, in searchTasks.

* How I found it:

- I traced the request handler before it called the repository. I found that the endpoint deliberately slept based on the search-term length, which caused an unnecessary delay even for an empty or short search.
- I also checked the request parameters and found that an unknown status could cause an exception, while invalid page values could cause the list slicing to fail.

* Root cause and fix:

- The request handler contained a debug-style delay and did not validate all incoming parameters before using them.
- I removed the delay and console logging. I added validation so unsupported status values and invalid page values return HTTP 400.
- Page sizes are now restricted to 1 through 100. I also changed the page offset calculation to use long so a very large page does not cause integer overflow before the results are sliced.

* How I checked it:

- One empty-search request returned HTTP 200 in about 0.009 seconds. This was a smoke check, not a performance benchmark.

* Invalid input checks:

- Unknown status → HTTP 400
- page = 0 → HTTP 400
- pageSize = 0 → HTTP 400
- pageSize = 101 → HTTP 400

→ I also checked that page size 5 and 100 were accepted

→ For pagination, page=1 & page size=5 returned task IDs 49-45, while page=2 & page size=5 returned 44-40, confirming that the pages return different consecutive results.

→ Finally, page=214748364 & page size=100 returned HTTP 200 with no items instead of crashing.

* Why this validates it:

→ The empty search smoke check is consistent with the previous delay having been removed. The other checks show that the endpoint rejects invalid filters and page values, correctly separates pagination results, and safely handles a very large page number.

Fix#3: Preventing stale Requests and Fixing Loading Errors

* Location:

→ frontend/src/api.js, frontend/src/hooks/useTasks.js, and frontend/src/App.jsx.

* How I found it:

→ I traced the request handling and effect cleanup in the frontend hook. I then used the page to search, change the status filter, move between pages, and recover from a temporary backend outage.

* Root cause and fix:

→ The frontend requests did not use an abort signal, so an older request could finish after a newer request and overwrite the latest

results.

→ I also found that previous errors were not cleared when retrying, failed requests could leave that page stuck in a loading state, and changing the search or status did not reset the page number.

→ I fixed this by using an AbortController with fetch and cancelling the previous request when the effect is cleaned up after the search or filters change. Abort errors are ignored, and finally ensures loading ends for the active request.

→ I also clear the previous error which when a new request starts and reset the page to 1 whenever the search or status changes.

* How I checked it:

→ Searching mobile showed two matches. Selecting Open showed task 1, while selecting Done showed "No tasks found."

→ I then cleared the filters, moved to page 2 of 5, and selected Open. The page reset to page 1 of 4, confirming that changing the filter resets pagination.

→ I stopped the backend and changed the search. The page showed an error instead of remaining stuck on "Loading." After restarting the backend, searching mobile cleared the old error and showed task 1 again.

* Slow-request race check:

→ I temporarily delayed requests for the exact query slow by 2.5

seconds. I searched slow and immediately changed the input to mobile.

→ The slow request was aborted, while the mobile request completed. After waiting 3 seconds, the query was still mobile and the task table still showed the two mobile results.

→ I restored the original fetch function after the test, so the delay was not added to the application.

* Why this validates it:

→ These checks cover filter results, resetting pagination when a filter changes, ending the loading state after a failure, clearing the old error after a subsequent successful request, and preventing an older request from replacing the latest results.

Fix# 4: Improving API Error Messages and Adding Retry

* Location:

→ frontend/src/components/TaskTable.jsx, frontend/src/hooks/useTasks.js, frontend/src/App.jsx, and the error styles in frontend/src/styles.css.

* How I found it:

→ I stopped the backend and searched in the UI. The existing error view showed a raw error message and did not provide any way to retry the same request.

* Root cause and fix:

→ The table displayed the fetch error string directly, which was not very helpful to the user. The hook also only made a request when

the search, status, or page changed, so there was no way to repeat the same request after an error.

→ I changed the error view to show a clear message:

~~"We couldn't"~~

"We couldn't load the tasks. Check your connection and try again."

→ I also added a Retry button. The hook uses a retry counter as an effect dependency, so clicking Retry triggers a new request with the current search, status, and page.

→ I also added a clear keyboard focus outline to the Retry button.

* How I checked it:

→ With the backend stopped, I searched api. The page displayed the new error message and a Retry button, while the search field still contained api.

→ I then restarted the backend and clicked Retry without changing api in the search field. The page showed "Loading tasks..." and then displayed tasks matching api.

* Why this validates it:

→ The error message is now understandable, the current search is preserved, and the Retry button makes a new request using the same search after the backend becomes available again.

Fix#5: Adding Search Completion and Keyboard Shortcut

* Location:

→ frontend/src/components/SearchBar.jsx, frontend/src/App.jsx, and frontend/src/styles.css.

* How I found it:

→ I checked the search input and found that it only accepted typed text. There was no inline suggestions to help complete a task title and no keyboard shortcut to quickly focus the search field.

* Root cause and fix:

→ The search bar did not provide any completion or quick keyboard access.

→ I added an inline completion that shows a gray suggestion based on task titles that start with the typed text, using titles already loaded on the current page.

→ The suggestion can be accepted with Tab or Right Arrow when the cursor is at the end of the input.

→ I also added a keyboard shortcut using Ctrl+K on Windows/Linux and ⌘K on macOS to focus the search field. The shortcut hint changes based on the platform.

→ This uses the existing task data, so new dependency or API endpoint was required.

* How I checked it:

→ I typed Im into the search field. A gray suggestion for "Implementation rate limit per user", appeared and pressing Tab accepted the suggestion.

→ I then typed F1. Pressing Right Arrow accepted "Fix search not clearing on navigation."

→ I also focused the status dropdown and tested Ctrl+K. The shortcut focused the search field and prevented the browser's default action.

→ The page displayed Ctrl+K as the shortcut hint in Linux and Windows.

* Limitation:

→ The suggestion only use task titles from the currently loaded page, so at most 10 titles are considered at a time.

Validation Note:

→ I did not run the automated test suites. I manually checked the changes using the browser and curl requests.

→ I updated the Oracle reference package for consistency, but did not run it because the local setup uses H2.